

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

Asignatura: Criptografía

Profesor: Dr. Alfonso Francisco de Abiega L'Eglise

Proyecto Final

Implementación de Mecanismos Criptográficos en un Sistema Web de Gestión de Servicio Social

Integrantes: Álvarez López Carlos Manuel
Figueroa Ruiz Carolina
Hernández Gallardo Daniel Alonso
Jiménez Ortiz Sebastián
López Flores Isaac
López Tavera Alexa Fernanda
Nava Santiago Erick

Ciudad Universitaria, Cd. Mx.

26 de mayo de 2026

Índice

1. Introducción	2
2. Planteamiento del Problema	2
3. Arquitectura General del Sistema	3
3.1. Diagrama de Bloques y Flujo de Información	4
4. Dependencias utilizadas en el backend	4
5. Base de datos	5
6. Implementación del Esquema Criptográfico	6
7. Manual de Despliegue y Pruebas	10
7.1. Requisitos previos	10
7.2. Instrucciones de Despliegue	11
7.3. Pruebas básicas del sistema	12
7.4. Pruebas de seguridad del sistema	14
7.5. Guía de Pruebas Criptográficas (API Interactiva)	17
7.6. Errores comunes durante la ejecución	18
8. Repositorio de Código Fuente	18
9. Conclusiones	18

1. Introducción

La criptografía ha sido una herramienta fundamental para la protección de la información desde mucho antes del desarrollo de los sistemas digitales modernos. En sus primeras aplicaciones, se utilizaba principalmente en contextos militares, diplomáticos y gubernamentales, donde era necesario ocultar mensajes estratégicos para que solo pudieran ser comprendidos por personas autorizadas. Sin embargo, con el avance de la tecnología y la expansión de las comunicaciones electrónicas, la criptografía dejó de limitarse al intercambio secreto de mensajes y comenzó a formar parte esencial de la seguridad informática. Actualmente, se entiende como una disciplina que proporciona mecanismos para proteger la confidencialidad, integridad, autenticación y no repudio de la información, propiedades indispensables para que los sistemas digitales puedan operar de forma confiable.

En la actualidad, la mayoría de las actividades cotidianas dependen de plataformas digitales que almacenan, procesan y transmiten datos sensibles. El acceso a cuentas personales, las operaciones bancarias, la comunicación en línea, los expedientes médicos, los documentos institucionales y los registros académicos requieren mecanismos de seguridad que eviten el uso indebido de la información. En este contexto, la criptografía no solo sirve para cifrar datos, sino también para proteger contraseñas, verificar identidades, asegurar documentos, detectar alteraciones y reducir el impacto de posibles ataques o accesos no autorizados. Por esta razón, su aplicación es cada vez más importante en el diseño de sistemas que manejan información personal, ya que permite implementar controles técnicos que fortalecen la privacidad y la confianza de los usuarios.

Un ejemplo claro de esta necesidad se encuentra en los sistemas de gestión escolar, ya que estos trabajan con datos personales e institucionales como nombres, matrículas, correos, teléfonos, direcciones, documentos y registros administrativos. La protección de esta información es importante no solo por razones técnicas, sino también porque forma parte del derecho de las personas a que sus datos sean tratados de manera legítima, controlada e informada [1]. En este proyecto se verá aplicada la criptografía dentro de un sistema de gestión escolar para el servicio social, con el propósito de demostrar cómo los principios criptográficos pueden integrarse en una solución real de software para proteger credenciales, datos sensibles, identificadores y documentos formales.

2. Planteamiento del Problema

La gestión del servicio social requiere un proceso ordenado, confiable y accesible para que los alumnos puedan consultar opciones disponibles, registrar su participación y dar seguimiento a su avance dentro de una institución o empresa receptora. Sin embargo, cuando este proceso se realiza de forma manual o con herramientas dispersas, pueden presentarse problemas como pérdida de información, duplicidad de registros, falta de control sobre las empresas disponibles, dificultad para validar datos de los alumnos y poca claridad en el seguimiento administrativo. Por esta razón, surge la necesidad de desarrollar un portal web que centralice la información relacionada con el servicio social y permita organizar el registro de alumnos,

empresas y actividades dentro de una misma plataforma.

El portal debe atender las necesidades específicas de distintos tipos de usuarios. Los alumnos requieren un espacio donde puedan registrarse, consultar el catálogo de empresas o programas disponibles, generar una identificación asociada a su matrícula y obtener documentos relacionados con su proceso. Las empresas necesitan registrar sus datos institucionales, publicar o administrar sus oportunidades de servicio social y visualizar información relevante de los alumnos inscritos. Por su parte, los administradores requieren herramientas para supervisar los registros, validar información, gestionar usuarios, controlar el catálogo de empresas y consultar datos generales del sistema mediante un panel de administración. De esta manera, el problema principal consiste en diseñar un sistema web que no solo facilite el registro y la gestión del servicio social, sino que también mantenga un control adecuado de la información de acuerdo con los roles que intervienen en el proceso.

Sin embargo, el portal no solo debe resolver la parte funcional del registro y la administración del servicio social, sino también incorporar mecanismos de seguridad que protejan la información manejada por el sistema. Esto implica aplicar técnicas criptográficas para resguardar credenciales, proteger datos sensibles, asegurar identificadores como la matrícula del alumno y mantener la integridad de documentos generados dentro de la plataforma. De esta forma, la criptografía se integra como una parte esencial del diseño del sistema, permitiendo que la gestión de la información sea más segura, confiable y adecuada para un entorno académico.

3. Arquitectura General del Sistema

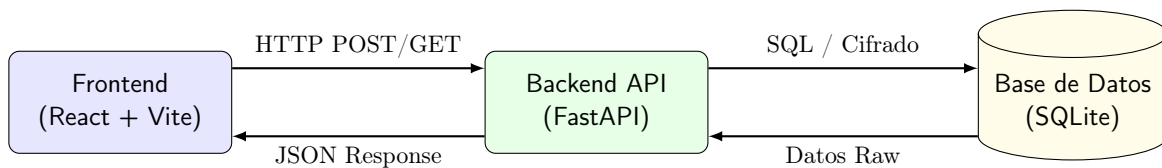
El sistema se desarrolló bajo una arquitectura web dividida en tres componentes principales: frontend, backend y base de datos. La interfaz de usuario fue implementada con React y Vite, permitiendo construir una aplicación dinámica, modular y fácil de actualizar. Desde esta capa, los usuarios pueden interactuar con las diferentes secciones del portal, como el panel de administración, el registro y consulta de alumnos, así como la gestión de empresas participantes en el servicio social. Esta interfaz funciona como el punto de entrada del sistema, ya que permite capturar, visualizar y enviar información hacia el servidor de acuerdo con el rol correspondiente.

El backend fue implementado con Python utilizando FastAPI, el cual actúa como el motor lógico del sistema. Esta capa recibe las solicitudes generadas desde el frontend, procesa la información, valida los datos, aplica las operaciones necesarias y se comunica con la base de datos SQLite para guardar o consultar registros. Además, en el backend se concentran los procesos relacionados con la seguridad y la criptografía, como el resguardo de credenciales, la protección de datos sensibles y la generación de elementos seguros asociados al registro de los usuarios. De esta forma, el backend funciona como una capa intermedia entre la interfaz y la base de datos, evitando que el frontend tenga acceso directo a la información almacenada.

La base de datos SQLite se utiliza como el componente de almacenamiento del sistema, donde se conservan los registros de alumnos, empresas, administradores y datos asociados al servicio social. La comunicación entre las capas se realiza mediante solicitudes HTTP y respuestas en formato JSON, lo que permite que el frontend y el backend intercambien información de manera estructurada. En conjunto, esta arquitectura permite separar responsabilidades: el frontend se encarga de la presentación e interacción visual, el backend administra la lógica, validaciones y seguridad, mientras que la base de datos mantiene la persistencia de la información.

3.1. Diagrama de Bloques y Flujo de Información

A continuación se modela de forma simplificada la interacción e intercambio de información entre las capas del sistema:



4. Dependencias utilizadas en el backend

Para el desarrollo del backend se utilizaron distintas dependencias de Python que permiten construir la API, ejecutar el servidor local, validar la información recibida y aplicar los mecanismos criptográficos definidos para el sistema. Estas librerías se encuentran registradas en el archivo `dependencias.txt`, por lo que pueden instalarse de manera automática mediante el comando `pip install -r dependencias.txt`. A continuación, se describe la función principal de cada una dentro del proyecto:

- **fastapi**: es el framework utilizado para construir la API del sistema [2]. Permite definir los endpoints encargados del registro de alumnos, empresas y administradores, así como las rutas para iniciar sesión, consultar información y generar contratos. Esta dependencia funciona como la base principal del backend, ya que permite la comunicación entre el frontend desarrollado en React y la lógica del servidor implementada en Python.
- **uvicorn[standard]**: es el servidor ASGI utilizado para ejecutar la aplicación desarrollada con FastAPI. Gracias a esta dependencia, el backend puede levantarse de manera local mediante el comando `uvicorn main:app -reload`. La opción `-reload` resulta útil durante el desarrollo, ya que permite reiniciar automáticamente el servidor cuando se realizan cambios en el código.
- **cryptography**: es la librería utilizada para implementar mecanismos de cifrado dentro del sistema. En este proyecto se emplea principalmente para proteger datos sensibles mediante el esquema **Fernet**, evitando que información como nombres, teléfonos, correos o direcciones se almacene directamente en texto plano dentro de la base de datos. Su uso permite reforzar la confidencialidad de la información registrada.

- **pydantic**: se utiliza para definir modelos de datos y validar la información recibida por la API. Esta dependencia permite actualizar los datos enviados desde el frontend, asegurando que los registros de alumnos, empresas o administradores tengan el formato esperado antes de ser procesados por el backend. De esta manera, ayuda a reducir errores en la captura y manejo de información.
- **bcrypt**: es la dependencia encargada del resguardo seguro de contraseñas. Su función es transformar las contraseñas de los usuarios en cadenas hash irreversibles antes de almacenarlas en la base de datos. Esto evita que las contraseñas se guarden en texto plano. Además, **bcrypt** incorpora sal y un costo computacional configurable, lo cual fortalece la protección frente a ataques de fuerza bruta [3, 4].
- **qrcode[pil]**: se utiliza para generar códigos QR dentro del sistema. La opción **[pil]** permite trabajar con imágenes mediante Pillow, lo cual facilita la creación visual del código QR a partir de una cadena generada por el sistema. En este proyecto, el QR se utiliza como mecanismo de identificación del alumno, evitando exponer directamente la matrícula en texto claro.
- **dilithium-py**: es la librería utilizada para implementar firmas digitales post-cuánticas basadas en el esquema CRYSTALS-Dilithium. En el sistema se emplea para generar la firma digital asociada a los contratos, reforzando la integridad del documento y permitiendo validar que el contrato corresponde al contenido generado originalmente.

También es importante mencionar que SQLite no aparece como una dependencia externa dentro del archivo de instalación, ya que Python incluye soporte para SQLite mediante el módulo **sqlite3**. Por esta razón, la base de datos puede utilizarse directamente desde el backend sin instalar una librería adicional.

5. Base de datos

El sistema utiliza una base de datos SQLite llamada **sistema_cripto.db**, la cual se encarga de almacenar la información necesaria para el funcionamiento del portal de servicio social. Esta base de datos permite conservar los registros de alumnos, empresas, administradores y contratos generados dentro del sistema. Al ser SQLite una base de datos ligera y local, resulta adecuada para el desarrollo y prueba del proyecto, ya que no requiere la instalación de un servidor externo y puede integrarse directamente con el backend desarrollado en Python.

Dentro de la base de datos se manejan tablas principales como **alumnos**, **empresas**, **administradores** y **contratos**. En estas tablas se almacenan los datos correspondientes a cada rol del sistema, así como la información relacionada con los documentos generados. Sin embargo, debido a que el proyecto tiene un enfoque criptográfico, la base de datos no almacena toda la información en texto plano. Las contraseñas se guardan como hashes mediante **bcrypt**, mientras que ciertos datos sensibles se almacenan cifrados para evitar que puedan leerse directamente al abrir el archivo de la base de datos.

La tabla de contratos almacena el contenido del documento generado para el alumno, junto con su firma digital y el estado de validación correspondiente. Esto permite mantener

una relación entre el alumno registrado y el contrato formal asociado a su servicio social. De esta manera, la base de datos no solo cumple la función de guardar información administrativa, sino que también conserva los elementos criptográficos necesarios para comprobar la integridad y seguridad de los datos manejados por el sistema.

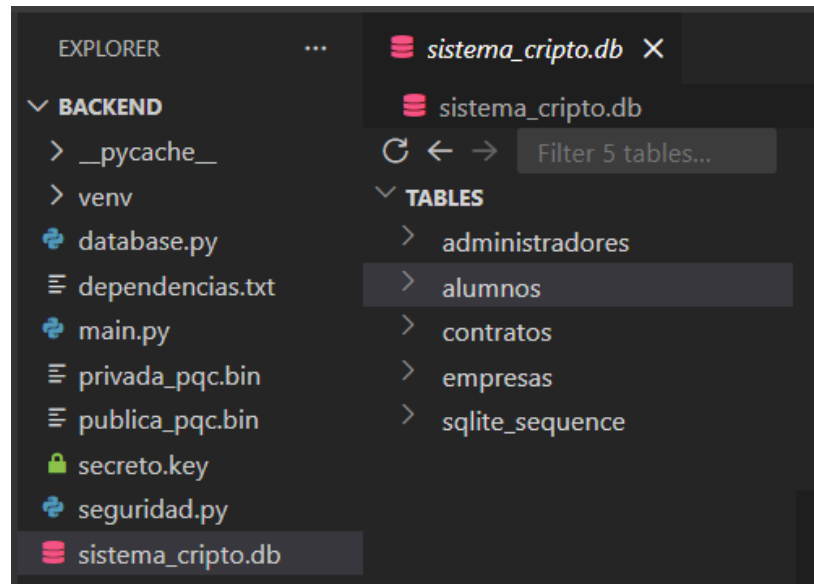


Figura 1: Visualización de las tablas principales de la base de datos `sistema_cripto.db`.

6. Implementación del Esquema Criptográfico

La seguridad del sistema se implementó principalmente en la capa del backend, ya que esta parte de la arquitectura es la encargada de recibir los datos del frontend, procesarlos, aplicar los mecanismos criptográficos correspondientes y comunicarse con la base de datos SQLite. De esta manera, la lógica de protección no queda expuesta directamente al usuario, sino que se concentra en el servidor mediante funciones específicas para el manejo de contraseñas, cifrado de datos sensibles, generación de códigos QR seguros y firma digital de contratos. Esta separación permite que el frontend funcione únicamente como interfaz de captura y consulta, mientras que el backend mantiene el control de los procesos críticos de seguridad.

El esquema criptográfico implementado busca atender cuatro necesidades principales del sistema: proteger las credenciales de acceso, resguardar información personal, evitar la exposición directa de identificadores escolares y asegurar la integridad de documentos formales. Cada una de estas necesidades se resolvió mediante un mecanismo distinto, de acuerdo con el tipo de información que se desea proteger y el uso que tendrá dentro de la plataforma.

- **Resguardo de Credenciales:** Para proteger las contraseñas de alumnos, empresas y administradores, el sistema utiliza un esquema de hashing unidireccional mediante el algoritmo `bcrypt`. Este mecanismo permite transformar la contraseña original en una cadena hash que no puede revertirse para obtener el valor inicial. Por esta razón,

la contraseña ingresada por el usuario nunca se almacena directamente en la base de datos, sino que se guarda únicamente su representación procesada.

El uso de `bcrypt` resulta adecuado para el almacenamiento de contraseñas porque incorpora un costo computacional configurable y el uso de sal, lo cual dificulta ataques de fuerza bruta o el uso de tablas precalculadas. Cuando un usuario intenta iniciar sesión, el sistema no compara la contraseña en texto plano contra un valor almacenado, sino que valida la contraseña ingresada contra el hash guardado en la base de datos. Si ambos valores corresponden, se permite el acceso; de lo contrario, la autenticación es rechazada.

- **Privacidad de Datos Sensibles:** Además de las contraseñas, el sistema maneja datos personales e institucionales que requieren protección, como nombres, apellidos, correos electrónicos, teléfonos y direcciones. Para resguardar esta información, se implementó cifrado simétrico mediante el esquema **Fernet**. Se eligió esta herramienta porque implementa bajo el capó el estándar AES en modo CBC con un código de autenticación de mensajes (HMAC) basado en SHA-256. Esto garantiza no solo la confidencialidad de los datos antes de almacenarlos en la base de datos, sino también su integridad, asegurando que la información no pueda ser leída ni manipulada sin la llave correspondiente.

El cifrado simétrico funciona mediante una llave secreta compartida por el sistema. Esta llave se utiliza tanto para cifrar como para descifrar la información, por lo que debe mantenerse protegida dentro del backend y no exponerse en el frontend. En este proyecto, la llave se encuentra separada de los datos almacenados, permitiendo que la información guardada en SQLite no sea legible directamente si alguien abre el archivo de la base de datos sin autorización.

En la práctica, cuando un alumno o empresa registra información sensible, el backend recibe los datos, aplica el proceso de cifrado y posteriormente guarda el resultado cifrado en la base de datos. Al consultar la información desde el portal, el backend realiza el proceso inverso: recupera los datos cifrados, los descifra con la llave correspondiente y los envía al frontend para su visualización. De esta forma, la base de datos no almacena los datos personales en texto claro, sino como valores protegidos.

En el siguiente fragmento se muestra la implementación técnica de este mecanismo. Se desarrolló una función para generar y almacenar de forma persistente la llave secreta si esta no existe, garantizando que siempre se utilice la misma para las operaciones. Posteriormente, se instanció el cifrador y se definieron las funciones de encriptación y desencriptación, las cuales se mandan a llamar desde los endpoints al procesar la información del usuario:

```
1 def obtenerLlaveFernet():
2     if not os.path.exists("secreto.key"):
3         llave = Fernet.generate_key()
4         with open("secreto.key", "wb") as archivoLlave:
5             archivoLlave.write(llave)
6         with open("secreto.key", "rb") as archivoLlave:
7             return archivoLlave.read()
8
9     cifrador = Fernet(obtenerLlaveFernet())
```



```

10
11     def encriptarDato(texto: str) -> bytes:
12         return cifrador.encrypt(texto.encode('utf-8'))
13
14     def desencriptarDato(textoCifrado: bytes) -> str:
15         return cifrador.decrypt(textoCifrado).decode('utf-8')
16

```

Listing 1: Implementación del cifrado simétrico con Fernet para protección de datos personales.

- **Identificación Segura (QR):** Para la identificación de los alumnos, el sistema utiliza códigos QR generados a partir de la matrícula universitaria. Sin embargo, por motivos de privacidad, la matrícula no se inserta directamente en el QR en texto plano. Antes de generar el código, el backend aplica un proceso de protección sobre el identificador del alumno, de manera que el QR contenga una cadena cifrada o codificada, y no el dato escolar visible.

Este procedimiento evita que cualquier persona pueda escanear el QR y obtener directamente la matrícula del alumno. Aunque el código pueda ser leído por una cámara o una aplicación externa, el resultado obtenido no corresponde al identificador original, sino a una cadena extensa que solo puede ser interpretada correctamente por el sistema. La codificación en Base64 permite representar el resultado en un formato compatible con el QR, mientras que la protección criptográfica evita la exposición directa de la información.

De esta manera, el QR funciona como un mecanismo de identificación segura dentro del sistema. Su objetivo no es ocultar la existencia del alumno, sino evitar que el identificador escolar sea visible de forma inmediata para usuarios externos. Esto permite utilizar el código como elemento práctico de consulta o validación sin comprometer directamente la privacidad del alumno.

Para llevar esto a cabo a nivel de código, la función encargada de generar el QR primero recibe la matrícula y la pasa por el proceso de cifrado simétrico previamente definido. Después, el resultado en bytes se codifica en Base64 para asegurar su compatibilidad como texto plano dentro del código QR. Finalmente, se genera la imagen utilizando la paquetería correspondiente y se devuelve como una cadena en formato URI para que el frontend pueda renderizarla directamente sin necesidad de almacenar la imagen física en el servidor:

```

1     def generarQrCifrado(matricula: str) -> str:
2         matriculaCifradaBytes = encriptarDato(matricula)
3         matriculaCifradaTexto = base64.b64encode(matriculaCifradaBytes
4             ).decode('utf-8')
5
6         qr = qrcode.QRCode(version=1, box_size=10, border=4)
7         qr.add_data(matriculaCifradaTexto)
8         qr.make(fit=True)
9
10        imagenQr = qr.make_image(fill_color="black", back_color="white
11            ")

```

```

10
11     buffer = BytesIO()
12     imagenQr.save(buffer, format="PNG")
13     imagenBase64 = base64.b64encode(buffer.getvalue()).decode("utf
-8")
14
15     return f"data:image/png;base64,{imagenBase64}"
16

```

Listing 2: Generación de código QR seguro con identificador cifrado.

- **Integridad de Documentos Formales:** El sistema también contempla la generación de contratos relacionados con el proceso de servicio social. Estos documentos tienen importancia administrativa, ya que formalizan la relación entre el alumno y la institución o empresa donde realizará sus actividades. Por esta razón, no basta con generar el contenido del contrato; también es necesario proteger su integridad y asociarlo con una evidencia criptográfica que permita validar que el documento no ha sido alterado.

Para atender esta necesidad, el sistema genera una firma digital asociada al contrato. En este proyecto se sustituyeron los métodos clásicos vulnerables al algoritmo de Shor (como RSA o ECC) por un esquema de firma post-cuántica basado en CRYSTALS-Dilithium [5]. Se optó por este esquema debido a que ha sido seleccionado por el NIST como el estándar principal para firmas digitales (FIPS 204), basando su seguridad en la dificultad matemática de los problemas sobre retículos (lattices). Este enfoque permite firmar el contenido del documento y producir una cadena criptográfica vinculada a dicho contenido que resulta computacionalmente segura incluso frente a futuros ataques con computación cuántica. Esta firma se almacena junto con el contrato y sirve como evidencia de que el documento fue generado por el sistema bajo un proceso controlado.

Cuando se consulta el contrato desde el backend, el sistema devuelve el contenido del documento, el estado de la firma y la firma digital generada. El campo `estadoFirma` permite indicar si la firma asociada al contrato es válida, mientras que el campo `firmaDigital` contiene la cadena resultante del proceso criptográfico. Esta implementación refuerza la integridad del documento, ya que cualquier modificación no autorizada sobre el contenido del contrato debería invalidar la correspondencia con su firma digital.

A nivel de implementación, el sistema inicializa las llaves públicas y privadas en formato binario al momento de arrancar el servidor. La librería `dilithium-py` se utiliza tanto para generar la firma a partir del hash del documento y la llave privada, como para ejecutar la función de verificación matemática (`verify`) con la llave pública. Este enfoque post-cuántico asegura que la validación de la integridad del documento sea resistente incluso ante futuros ataques con computación cuántica:

```

1     def inicializarLlavesPQC():
2         if not os.path.exists("privada_pqc.bin") or not os.path.exists
("publica_pqc.bin"):
3             llavePublica, llavePrivada = Dilithium2.keygen()
4
5         with open("privada_pqc.bin", "wb") as f:

```

```

6         f.write(llavePrivada)
7
8         with open("publica_pqc.bin", "wb") as f:
9             f.write(llavePublica)
10
11     def cargarLlavePrivadaPQC() -> bytes:
12         with open("privada_pqc.bin", "rb") as f:
13             return f.read()
14
15     def cargarLlavePublicaPQC() -> bytes:
16         with open("publica_pqc.bin", "rb") as f:
17             return f.read()
18
19     def firmarDato(datoEncriptado: bytes) -> bytes:
20         llavePrivada = cargarLlavePrivadaPQC()
21         firma = Dilithium2.sign(llavePrivada, datoEncriptado)
22         return firma
23
24     def verificarFirma(datoEncriptado: bytes, firma: bytes) -> bool:
25         llavePublica = cargarLlavePublicaPQC()
26         try:
27             esValida = Dilithium2.verify(llavePublica, datoEncriptado,
28             firma)
29             return esValida
30         except Exception:
31             return False
32
33     inicializarLlavesPQC()

```

Listing 3: Generación y validación de firmas digitales post-cuánticas usando Dilithium2.

7. Manual de Despliegue y Pruebas

Para ejecutar el sistema de manera local, es necesario preparar dos entornos de trabajo: uno para el backend, desarrollado con Python y FastAPI, y otro para el frontend, desarrollado con React y Vite. El backend es el encargado de procesar la lógica del sistema, conectarse con la base de datos SQLite y ejecutar los mecanismos de seguridad implementados. El frontend, por su parte, permite visualizar e interactuar con el portal web desde el navegador. Por esta razón, primero debe iniciarse el servidor del backend y posteriormente el servidor del frontend.

7.1. Requisitos previos

Antes de iniciar con el despliegue local de la aplicación, es indispensable asegurar que el sistema operativo cuente con los siguientes entornos de ejecución instalados y configurados en las variables de entorno:

- **Python 3.10 o superior:** Necesario para ejecutar el servidor asíncrono y las librerías criptográficas.

- **Node.js (LTS):** Incluye el gestor de paquetes `npm`, requerido para la construcción de la interfaz de usuario.

7.2. Instrucciones de Despliegue

La arquitectura del sistema requiere la ejecución simultánea de dos servidores locales (Backend y Frontend). Para ello, se deben abrir dos terminales independientes en el sistema.

1. Configuración del Backend (Terminal 1)

Desde la terminal, se debe navegar a la carpeta del backend, inicializar el entorno virtual de Python e instalar las dependencias criptográficas requeridas para el servidor FastAPI:

```
1 # 1. Ingresar al directorio del backend
2 cd backend
3
4 # 2. Crear el entorno virtual
5 python -m venv venv
6
7 # 3. Activar el entorno virtual (En Windows con la Terminal)
8 .\venv\Scripts\activate
9
10 # 4. Instalar las dependencias del sistema
11 pip install -r dependencias.txt
12
13 # 5. Levantar el servidor de FastAPI
14 uvicorn main:app --reload
```

Listing 4: Comandos para inicializar el servidor backend.

El backend comenzará a operar localmente, exponiendo la API en la dirección: <http://127.0.0.1:8000>.

2. Configuración del Frontend (Terminal 2)

En una segunda terminal, se descargan los módulos de Node y se inicia el entorno de desarrollo de Vite para la interfaz en React:

```
1 # 1. Ingresar al directorio del frontend
2 cd ../frontend
3
4 # 2. Instalar las dependencias de React
5 npm install
6
7 # 3. Levantar el servidor de desarrollo de Vite
8 npm run dev
```

Listing 5: Comandos para inicializar la interfaz frontend.

Una vez finalizado, el portal web estará disponible en el navegador a través de la ruta: <http://localhost:5173>.

7.3. Pruebas básicas del sistema

Después de iniciar ambos servidores, se deben realizar pruebas básicas para verificar que el sistema funciona correctamente. Primero, se comprueba que el backend esté activo entrando a la ruta `/docs` de FastAPI. Si la documentación carga correctamente, significa que la API está funcionando. Después, se abre la dirección local del frontend y se revisa que la interfaz cargue sin errores visuales.

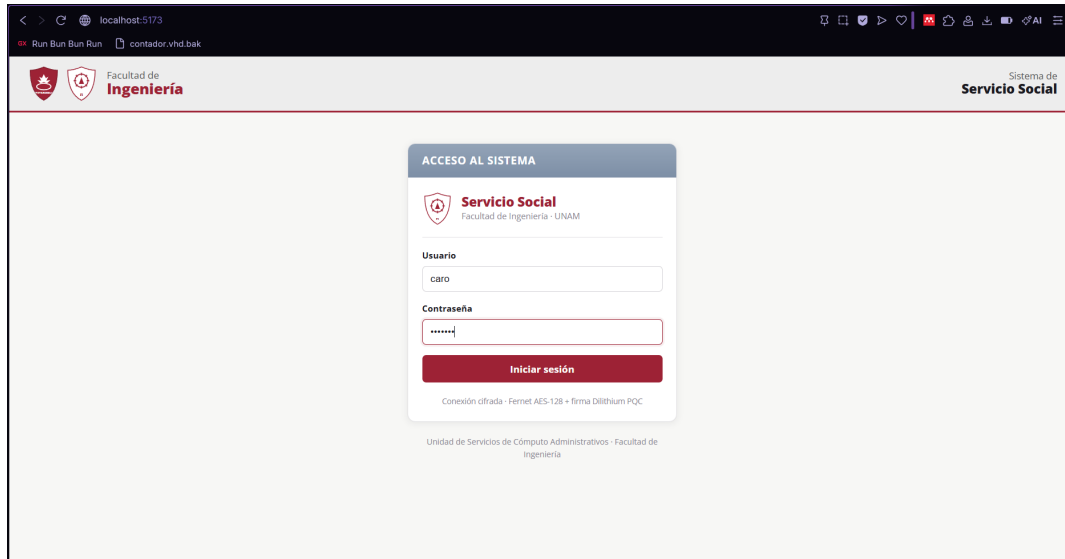


Figura 2: Interfaz principal del sistema de gestión de servicio social.

Posteriormente, se pueden realizar pruebas funcionales desde el portal web. En el módulo de alumnos, se puede verificar que sus datos y el QR que se genera se ven reflejados correctamente en el sistema:

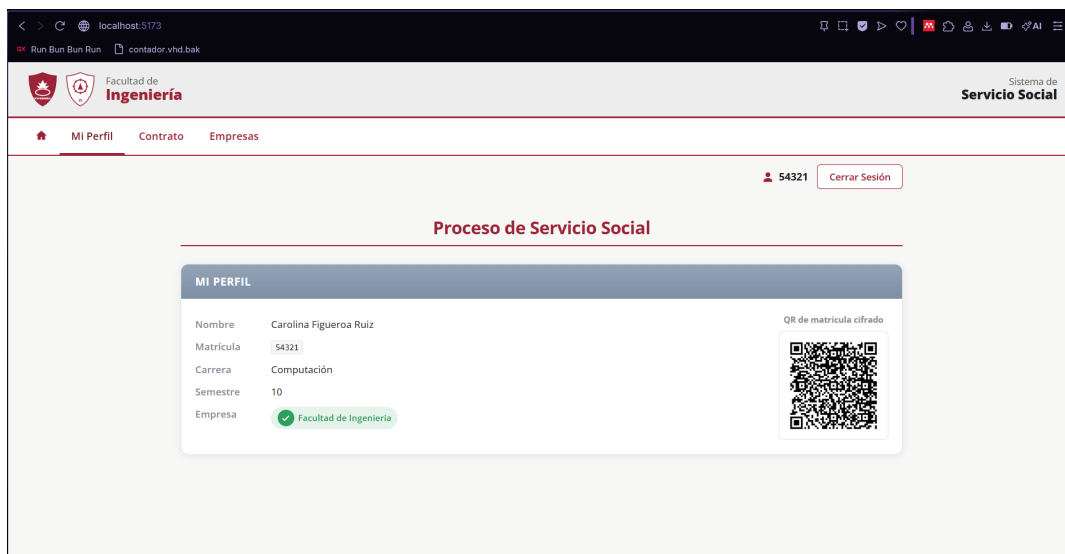


Figura 3: Interfaz del alumno con su QR de registro.

Así como la firma en el contrato:

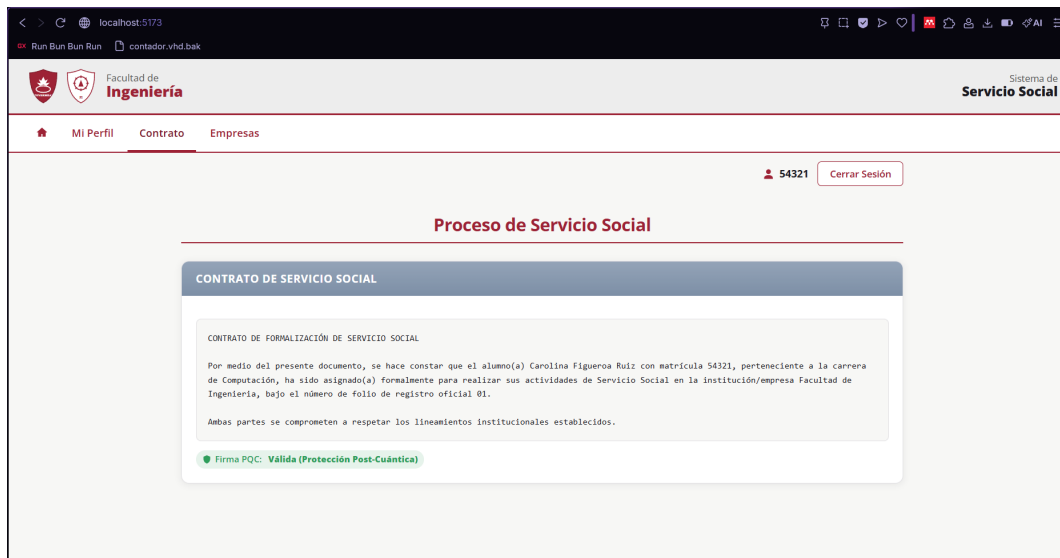


Figura 4: Firma PQC para el contrato con el alumno.

En el módulo de empresas, se puede comprobar que sea posible registrar una empresa con sus datos generales y que aparezca dentro del catálogo correspondiente:

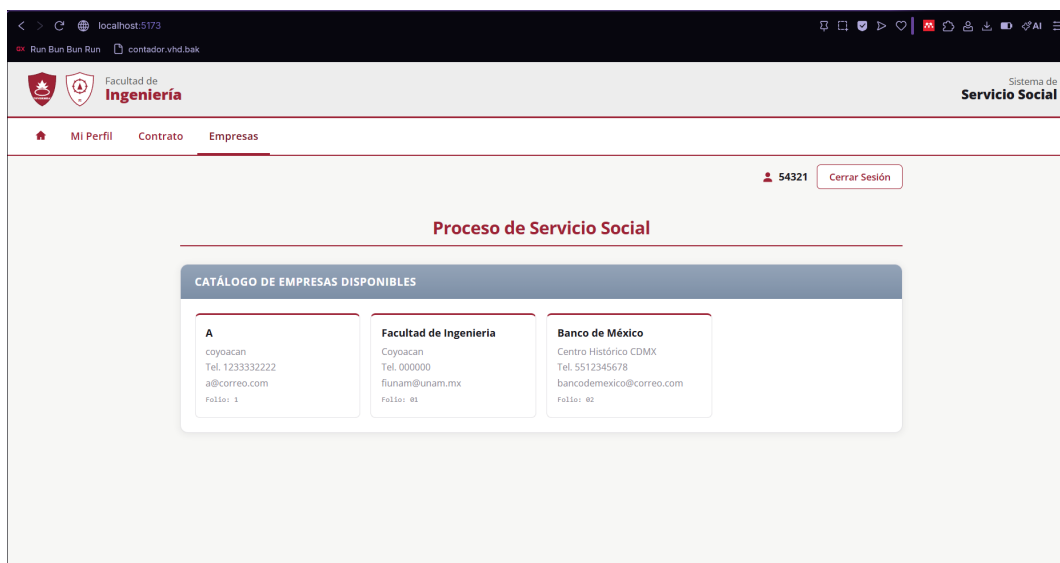


Figura 5: Catálogo con las empresas registradas.

En el panel de administración (user: admin - pass: admin123), se debe revisar que los contadores, tablas o secciones de resumen se actualicen de acuerdo con los registros realizados:

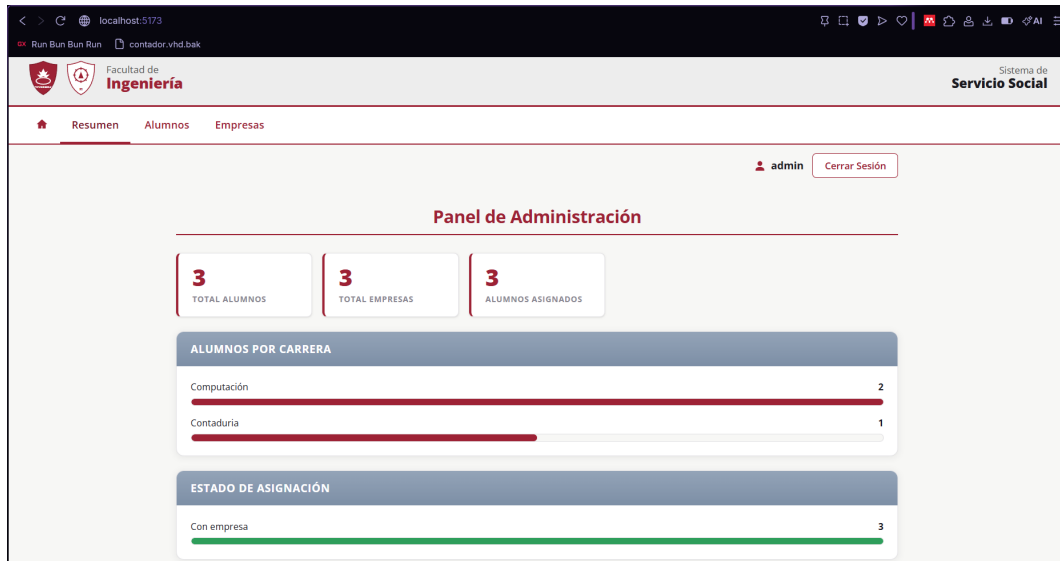


Figura 6: Interfaz del administrador con el registro y control de todo.

7.4. Pruebas de seguridad del sistema

Para validar el resguardo de credenciales, se revisó directamente la base de datos SQLite del sistema. En la tabla de alumnos se identificó el campo `contrasenaHash`, el cual almacena la contraseña procesada mediante `bcrypt`. El resultado observado muestra que las contraseñas no se guardan en texto plano, sino como una cadena hash con el prefijo característico `$2b$12$`. Esto confirma que el sistema no conserva la contraseña original del usuario y reduce el riesgo de exposición en caso de acceso no autorizado a la base de datos.

al...	matricula	usuario	contrasenaHash	nombreE...
1	12345	JPG12	\$2b\$12\$eKNuUDtpic8BwzIU1r0Z.t2Sb888Q7kC.4UoOIR...	100 bytes
2	9876	Alexa Tavera	\$2b\$12\$xCzQ/966OTp3rfx8B28QsOUogzNxtia6uDsuH/A...	100 bytes
3	54321	caro	\$2b\$12\$PF78u4ckHmcfqrVY8dUmQ.hty4Bdo90Th1rJXR...	100 bytes

Figura 7: Verificación del almacenamiento de contraseñas mediante hash bcrypt en la base de datos.

Para comprobar la generación de contratos, se utilizó el endpoint `POST/contratos/generar/{alumnoId}` desde la documentación automática de FastAPI. En esta prueba se ingresó el identificador de un alumno registrado en el sistema y se ejecutó la solicitud correspondiente. El servidor respondió con el código 200, indicando que la petición fue procesada correctamente. Como resultado, se obtuvo el contenido del contrato generado, asociado al alumno seleccionado y a su información de servicio social.

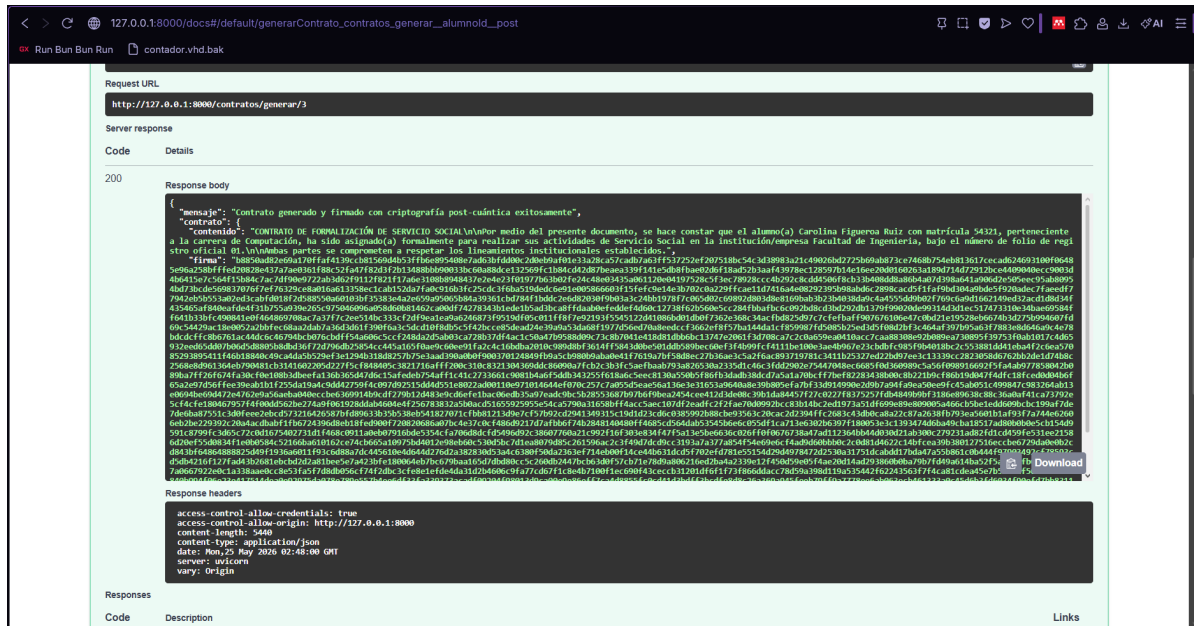


Figura 8: Generación exitosa del contrato mediante el endpoint correspondiente en FastAPI.

Para validar la protección criptográfica del contrato, se consultó el documento generado mediante el endpoint `GET /contratos/alumno/{alumnoId}`. En la respuesta del servidor se observó el contenido del contrato, el identificador del documento, el campo `estadoFirma` y la firma digital asociada. El sistema mostró el estado `Válida` (Protección Post-Cuántica), lo cual indica que el contrato fue firmado correctamente y que la firma se encuentra asociada al contenido del documento.

La presencia del campo `firmaDigital`, representado por una cadena extensa de caracteres, confirma que el sistema genera una evidencia criptográfica para el contrato. Esta firma permite reforzar la integridad del documento, ya que vincula el contenido generado con un valor criptográfico que puede utilizarse para detectar alteraciones o validar su autenticidad dentro del sistema.

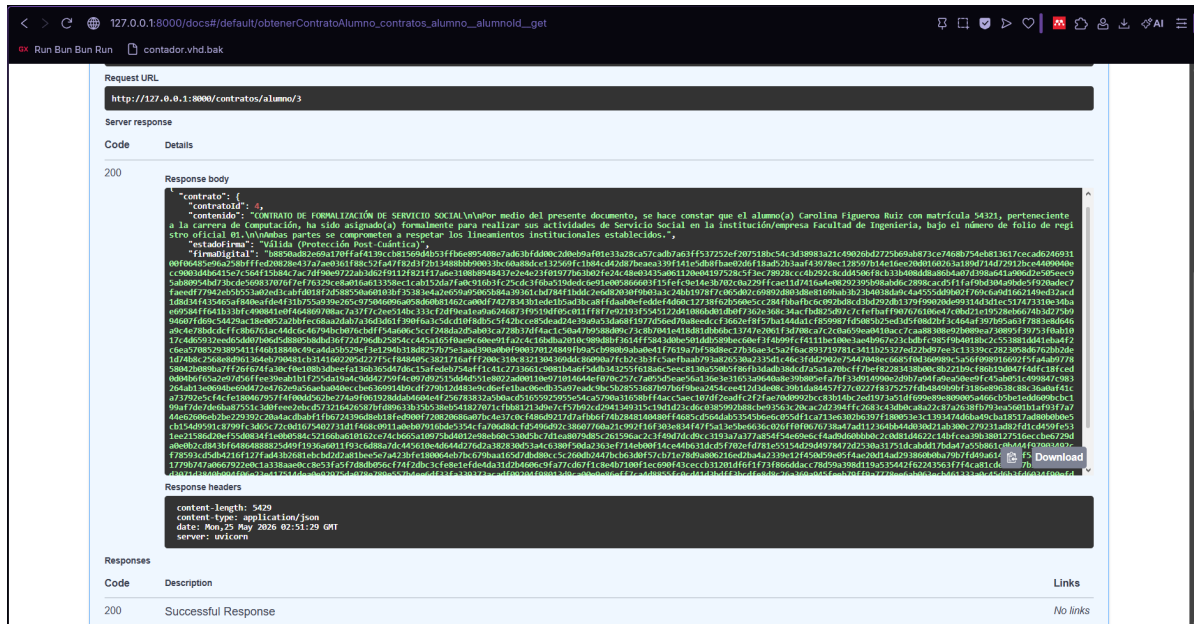


Figura 9: Consulta de contrato firmado con estado de firma válida y protección post-cuántica.

Al escanear el código, se obtuvo una cadena extensa de caracteres codificada, y no la matrícula original del alumno. Esto demuestra que el QR no almacena el identificador escolar de forma directamente legible. Por lo tanto, aunque una persona externa escanee el código, no puede obtener de manera inmediata la matrícula ni interpretar el dato sin el proceso correspondiente del sistema.

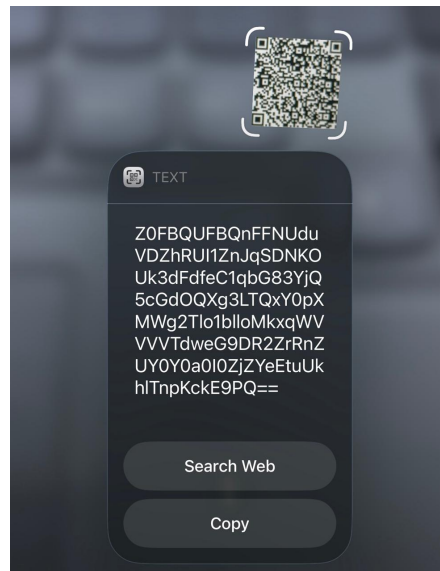


Figura 10: Resultado obtenido al escanear el código QR generado por el sistema.

Estas pruebas permiten comprobar que el sistema no solo funciona a nivel visual, sino que también aplica los mecanismos criptográficos definidos para proteger la información.

7.5. Guía de Pruebas Criptográficas (API Interactiva)

Dado que el motor criptográfico se encuentra completamente operativo en la capa del backend, la forma más eficiente de validar los flujos de seguridad y replicar los resultados anteriores es utilizando la interfaz interactiva automatizada de FastAPI (Swagger UI), disponible en <http://127.0.0.1:8000/docs>.

A continuación, se documenta el flujo de prueba recomendado para auditar los endpoints críticos del sistema mediante peticiones JSON:

Paso 1: Registro Seguro de Institución Receptora

Se ejecuta una petición POST en la ruta `/empresas/`. El sistema intercepta estos datos y cifra simétricamente la dirección y los datos de contacto utilizando **Fernet** antes de almacenarlos en disco. El campo `folioRegistro` cuenta con una restricción de unicidad para evitar duplicados.

```
1 {  
2   "nombre": "Ctrl+Alt+Del Team",  
3   "direccion": "Ciudad Universitaria, CDMX",  
4   "telefono": "5512345678",  
5   "correo": "contacto@ctrlaltdel.com",  
6   "folioRegistro": "FOLIO-001"  
7 }
```

Listing 6: Cuerpo de la petición para el registro de empresas.

Paso 2: Registro de Alumno y Hashing de Credenciales

Posteriormente, se utiliza la ruta POST `/alumnos/` para enviar los datos del estudiante. Durante este proceso, el backend genera el hash de la contraseña empleando **bcrypt** (mitigando ataques de fuerza bruta), encripta la matrícula para inyectarla en el código QR y protege la identidad del alumno.

```
1 {  
2   "matricula": "312345678",  
3   "usuario": "sebjimort",  
4   "contrasena": "mi_contrasena_segura",  
5   "nombre": "Sebastián",  
6   "apellidoPaterno": "Jiménez",  
7   "apellidoMaterno": "Ortiz",  
8   "carrera": "Ingeniería en Computación",  
9   "semestre": 8,  
10  "empresaId": 1  
11 }
```

Listing 7: Cuerpo de la petición para el registro de alumnos.

Paso 3: Generación de Contrato y Firma Post-Cuántica

Finalmente, se invoca la ruta POST `/contratos/generar/{alumnoId}`, ingresando el identificador numérico generado en el paso anterior.

Como resultado de esta operación, el servidor extrae los datos encriptados de la base de datos, los descifra dinámicamente en la memoria RAM para redactar el contrato y utiliza el algoritmo matemático **CRYSTALS-Dilithium2** (Estándar FIPS 204) junto con la llave privada para estampar un sello digital en formato hexadecimal. Este proceso garantiza la autenticidad e integridad inalterable del documento frente a vectores de ataque convencionales y cuánticos.

7.6. Errores comunes durante la ejecución

Durante la ejecución local del sistema pueden presentarse algunos errores comunes. Si PowerShell no permite activar el entorno virtual, normalmente se debe a una restricción en la ejecución de scripts, la cual puede solucionarse con el comando `Set-ExecutionPolicy`. Si aparece un error indicando que `python` no es reconocido, se debe verificar la instalación de Python o usar la ruta completa del ejecutable. Si el comando `pip install -r dependencias.txt` falla, se debe confirmar que el archivo `dependencias.txt` exista dentro de la carpeta del backend.

En el caso del frontend, si `npm` no es reconocido, significa que Node.js no está instalado o no fue agregado correctamente a las variables de entorno del sistema. Si `npm run dev` marca error, se recomienda ejecutar primero `npm install` para asegurar que las dependencias estén descargadas. Finalmente, si la interfaz carga pero no muestra datos o no permite registrar información, se debe revisar que el backend siga ejecutándose y que la URL configurada en el frontend coincida con la dirección local del servidor de FastAPI.

8. Repositorio de Código Fuente

El historial completo de commits, configuraciones y el código fuente del proyecto se encuentra alojado de forma pública en la siguiente dirección de GitHub:

<https://github.com/Saiko-E/Cripto>

9. Conclusiones

El desarrollo de este proyecto demostró de manera práctica que la integración de mecanismos de seguridad desde el diseño inicial de un sistema es una práctica fundamental en la ingeniería de software actual. Como equipo, logramos consolidar un portal web funcional para la gestión del servicio social que no solo cumple con sus objetivos administrativos, sino que protege activamente la privacidad y los datos sensibles de sus usuarios mediante esquemas criptográficos robustos.

A nivel técnico, la implementación de una arquitectura híbrida nos permitió encontrar un balance óptimo entre rendimiento y seguridad. Por un lado, el uso de herramientas consolidadas como `bcrypt` y el estándar AES a través del esquema `Fernet` resolvió las necesidades inmediatas de confidencialidad y resguardo de credenciales con alta eficiencia operacional y un bajo costo de procesamiento. Por otro lado, la audaz incorporación de firmas digitales

post-cuánticas basadas en CRYSTALS-Dilithium2 (FIPS 204) para los contratos dotó al sistema de un nivel de protección vanguardista, garantizando la integridad y el no repudio de los documentos formales frente a las amenazas computacionales del mañana.

Finalmente, superar el reto de conectar la complejidad de la teoría criptográfica con el desarrollo web full-stack representó una experiencia invaluable. Este proyecto nos exigió aplicar el rigor analítico y técnico adquirido a lo largo de nuestra formación en la facultad dentro de un escenario de aplicación real. Entregar una solución integral, que abarca desde la experiencia de usuario hasta la protección matemática de la información, consolida nuestro perfil como futuros profesionales y nos prepara para enfrentar con responsabilidad los grandes retos de ciberseguridad en la industria tecnológica.

Referencias

- [1] Cámara de Diputados del H. Congreso de la Unión, “Ley federal de protección de datos personales en posesión de los particulares.” <https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf>, 2025. Diario Oficial de la Federación.
- [2] S. Ramírez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production.” <https://fastapi.tiangolo.com/>, 2018.
- [3] OWASP Foundation, “Password storage cheat sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. s.f. Consultado el 24 de mayo de 2026.
- [4] W. Stallings, *Cryptography and Network Security: Principles and Practice*. Pearson, 8th ed., 2020.
- [5] National Institute of Standards and Technology (NIST), *FIPS 204: Draft Post-Quantum Cryptography Digital Signature Standard (CRYSTALS-Dilithium)*, 2024. Draft Standard.
- [6] E. Barker, “Guideline for using cryptographic standards in the federal government: Cryptographic mechanisms,” Tech. Rep. NIST Special Publication 800-175B Revision 1, National Institute of Standards and Technology, 2020. DOI: 10.6028/NIST.SP.800-175Br1.